

JAGIELLONIAN UNIVERSITY
IN KRAKOW

Lecture 6: Probability and Variational Autoencod
Latent Spaces, Conditional and Disentangled Generation
Digits and Blood Cells

Deep Learning for Scientific Discovery

Marcin Płodzień

Atomic Optics Department
Institute of Theoretical Physics
Jagiellonian University in Kraków



Doctoral School
of Exact and
Natural Sciences
Jagiellonian University

Recap: Understanding Data without Labels

- PCA and t-SNE: a few coordinates that describe each sample
- k-means and DBSCAN: groups of similar samples
- autoencoders: a learned compression into a latent space; the reconstruction error flags unusual samples
- anomaly detection: unusual blood cells and credit-card fraud flagged by a large reconstruction error

Today: learn the distribution of the data itself, and draw new samples from it.

1

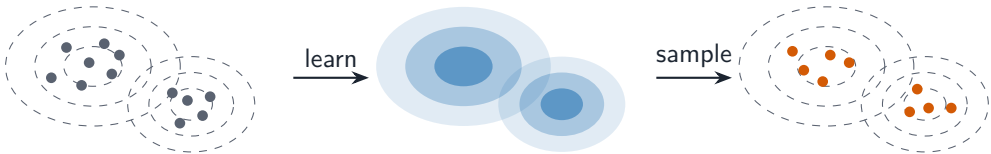
Generative Models

JAGIELLONIAN UNIVERSITY
IN KRAKOW

Learning a Probability Distribution

We have only data $\mathbf{x}_1, \ldots, \mathbf{x}_n$: images, spectra, spin configurations. No labels, nothing else. We assume they are independent samples of an unknown probability distribution $p(\mathbf{x})$.

Goal: a model $p_\theta(\mathbf{x})$ that we can *sample*: new $\mathbf{x} \sim p_\theta$ that look as if they came from $p(\mathbf{x})$ itself, with the same statistics (shapes, frequencies, correlations, energies), and that are not copies of the data.



data: samples of an unknown $p(\mathbf{x})$ (dashed)

model $p_\theta(\mathbf{x})$

new samples: where $p(\mathbf{x})$ puts its data

Supervised learning models $p(y \mid \mathbf{x})$; Lecture 5 found structure in the data; today we learn $p(\mathbf{x})$ itself.

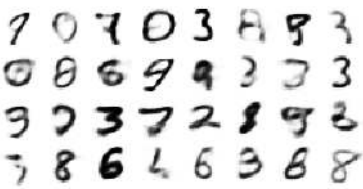
Generating New Data

A **generative model**

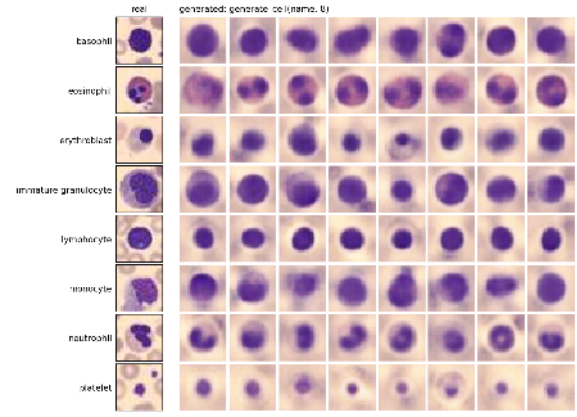
- learns an approximation $p_\theta(\mathbf{x})$ of $p(\mathbf{x})$ from the samples (θ: its parameters), by making the observed data probable under the model
- draws **new samples**: instances not in the data set, but that could have been
- can be checked: the generated samples must reproduce the statistics of the data (for spins, the physics: magnetisation, energy, correlations)

In science: new molecules and materials, protein structures, weather ensembles, fast surrogates of expensive simulations; and the learned p_θ itself as a description of the data.

Generated Digits and Blood Cells



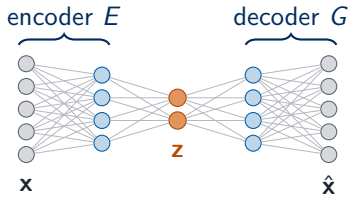
new digits: no image of the training set

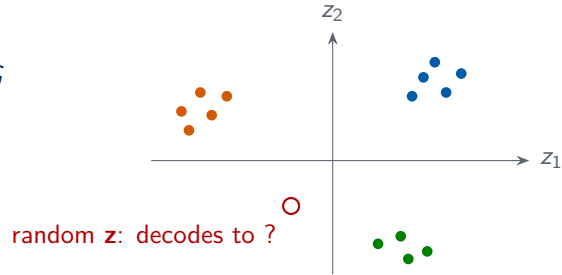


blood cells of a chosen type (first column: a real cell)

Both made by the models of this lecture, trained on a laptop CPU in minutes.

Autoencoder and Its Latent Space

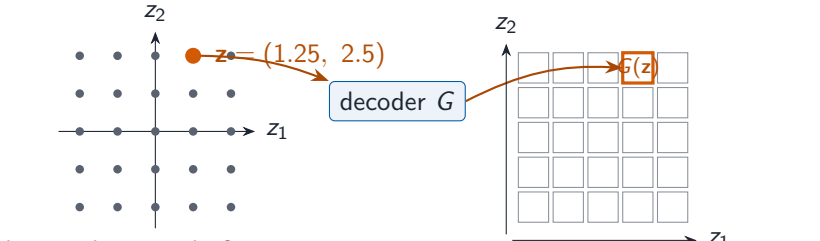




codes $E(\mathbf{x}_i)$ of training images (colour: digit)

- trained only to rebuild its inputs: each image becomes one point $\mathbf{z}_i = E(\mathbf{x}_i)$
- nothing fixes where the points lie: no known distribution to draw a new $\mathbf{z}$ from
- the decoder never saw the gaps: a $\mathbf{z}$ there decodes to something that is no digit

Latent Grid



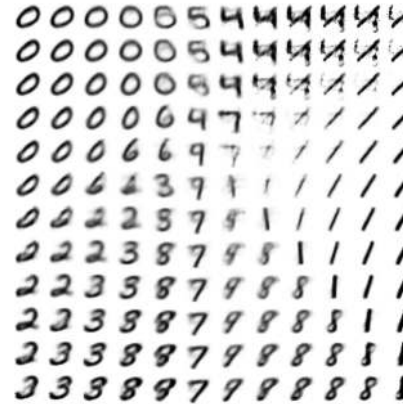
- a regular grid of latent points $\mathbf{z} = (z_1, z_2)$, $z_1, z_2 \in [-2.5, 2.5]$; each is decoded and its image placed at the same position (column: z_1, row: z_2)
- the sheet is a map of what the decoder produces everywhere in the latent plane, also where no training image was mapped

An Autoencoder Cannot Generate

Can the decoder of an autoencoder produce *new* digits? Pick a latent point $\mathbf{z}$ and compute $G(\mathbf{z})$.

- the autoencoder only learned to rebuild its training images; nothing tells us where in latent space the valid $\mathbf{z}$ are
- between and around the training codes there are regions that decode to images that are no digit

To generate, we need a latent space with a known distribution that we can sample from.



autoencoder with $d = 2$: images decoded from a regular grid of latent points

Generation with a Latent Space

The goal: new instances $\mathbf{x}$, drawn from (an approximation of) the distribution $p(\mathbf{x})$ of the data. The recipe of a latent-variable model:

$$\underbrace{\mathbf{z} \sim p(\mathbf{z})}_{\text{a random latent point}} \quad \longrightarrow \quad \underbrace{\mathbf{x} = G(\mathbf{z})}_{\text{decode it}}$$

It works only if two things hold:

1. $p(\mathbf{z})$ is *known* and easy to sample: we choose $p(\mathbf{z}) = \mathcal{N}(0, I)$
2. *every* $\mathbf{z}$ that $p(\mathbf{z})$ produces decodes to a plausible $\mathbf{x}$: the training data must fill the region where $p(\mathbf{z})$ puts its probability, without gaps

An autoencoder satisfies neither: its codes have an unknown, irregular distribution, with empty regions in between. The variational autoencoder is built to satisfy both, and it is trained so that its decoder becomes this generator G.

2
Probability



JAGIELLONIAN UNIVERSITY
IN KRAKOW

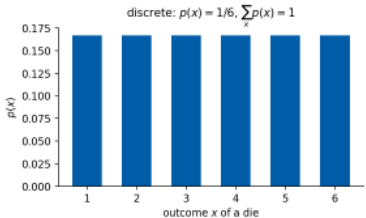
Discrete and Continuous Distributions

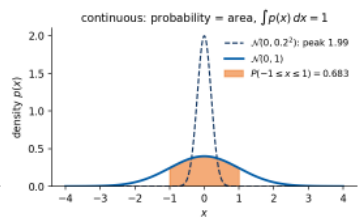
Discrete x (a die, a class, a spin): $p(x)$ is a probability,

$$0 \le p(x) \le 1, \qquad \sum_x p(x) = 1.$$

Continuous x (an intensity, a coordinate): $p(x)$ is a *density*; only areas are probabilities,

$$P(a \le x \le b) = \int_a^b p(x)\,dx, \qquad \int p(x)\,dx = 1$$



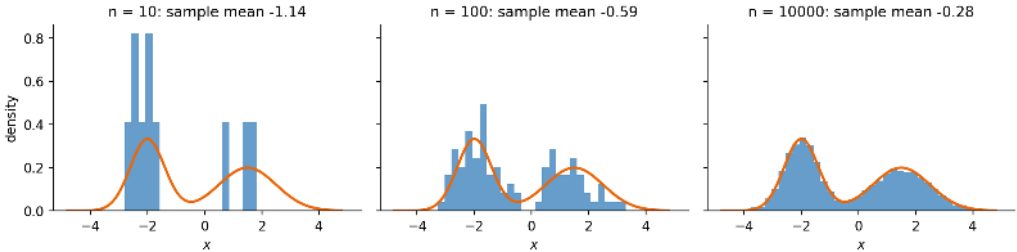


A density can exceed 1 (peak 1.99 for $\sigma = 0.2$). An MNIST image is a point $\mathbf{x} \in \mathbb{R}^{784}$, and $p(\mathbf{x})$ is a density in 784 dimensions.

Expectation and Averages over Samples

The **expectation** of a function f under p, and its estimate from samples $x_1, \ldots, x_n$ of p:

$$\mathbb{E}_{x\sim p}[f(x)] = \int p(x)\, f(x)\, dx \;\approx\; \frac{1}{n}\sum_{i=1}^{n} f(x_i), \qquad \text{std of the estimate} = \frac{\sigma_f}{\sqrt{n}}$$



Orange: $p(x) = \frac{1}{2}\mathcal{N}(x; -2, 0.6^2) + \frac{1}{2}\mathcal{N}(x; 1.5, 1)$, with $\mathbb{E}[x] = -0.25$; blue: histograms of n samples. Every loss in this lecture is an expectation, estimated as an average over a mini-batch (σ_f: standard deviation of $f(x)$ under p).

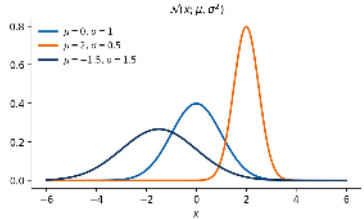
The Gaussian Distribution

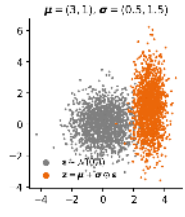
One dimension, mean μ, standard deviation σ:

$$\mathcal{N}(x; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-(x-\mu)^2/2\sigma^2}$$

d independent coordinates; $\mathcal{N}(0, I)$: all $\mu_k = 0$, $\sigma_k = 1$:

$$\mathcal{N}(\mathbf{z}; \boldsymbol{\mu}, \operatorname{diag} \boldsymbol{\sigma}^2) = \prod_{k=1}^{d} \mathcal{N}(z_k; \mu_k, \sigma_k^2)$$



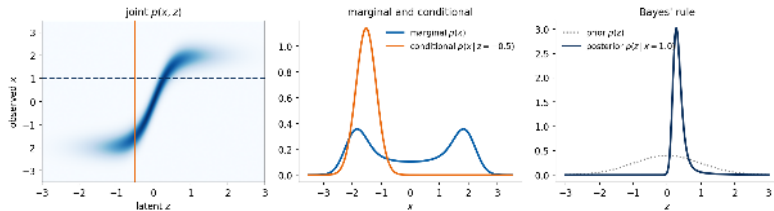


Sampling: draw $\varepsilon \sim \mathcal{N}(0, I)$, then $\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \varepsilon$ ($\odot$: element-wise product) is a sample of $\mathcal{N}(\boldsymbol{\mu}, \operatorname{diag} \boldsymbol{\sigma}^2)$: a shift and a stretch.

Joint, Conditional and Marginal Distributions

An observed x and a hidden (latent) z, with joint distribution $p(x, z)$:

$$\underbrace{p(x) = \int p(x, z)\, dz}_{\text{marginal}} \qquad \underbrace{p(x, z) = p(x \mid z)\, p(z)}_{\text{conditional, product rule}} \qquad \underbrace{p(z \mid x) = \frac{p(x \mid z)\, p(z)}{p(x)}}_{\text{Bayes' rule: posterior}}$$



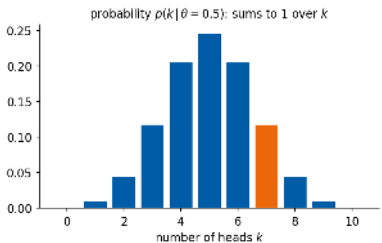
$p(z)$: the *prior*, what we know about z before seeing x; $p(z \mid x)$: the *posterior*, after seeing x. Here $z \sim \mathcal{N}(0, 1)$, $x \mid z \sim \mathcal{N}\big(2 \tanh(2z), 0.35^2\big)$: a two-peaked $p(x)$; observing $x = 1$ narrows the prior (std 1) to a posterior with std 0.25.

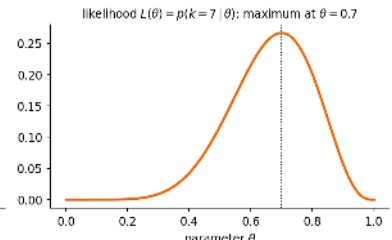
Probability and Likelihood

A model $p(x \mid \theta)$: the probability of data x when the parameters are θ. One formula, read in two ways:

probability: θ fixed, x varies; summed over all outcomes x it gives 1

likelihood $L(\theta) = p(x_{\text{obs}} \mid \theta)$: data fixed, θ varies; not a distribution over θ





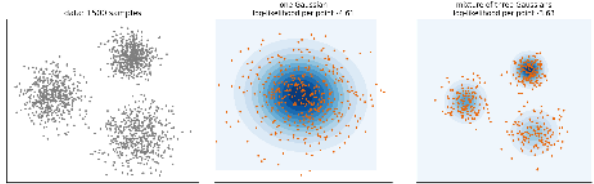
A coin with probability θ of heads, tossed 10 times: $p(k \mid \theta) = \binom{10}{k}\theta^k(1-\theta)^{10-k}$; $p(7 \mid 0.5) = 0.117$. With 7 heads observed, $L(\theta)$ peaks at $\theta = 0.7$, and $\int_0^1 L\, d\theta = 1/11$, not 1.

Maximum Likelihood

Independent samples $\mathbf{x}_1, \ldots, \mathbf{x}_n$; a model $p_\theta(\mathbf{x})$. The likelihood of all the data, and its logarithm:

$$L(\theta) = \prod_{i=1}^{n} p_\theta(\mathbf{x}_i), \qquad \ell(\theta) = \sum_{i=1}^{n} \log p_\theta(\mathbf{x}_i), \qquad \theta^* = \arg\max_\theta \ell(\theta)$$

The logarithm turns the product into a sum and does not move the maximum. One Gaussian: θ^* is the sample mean and variance (with $1/n$).



1500 points from three groups; blue: fitted density, orange: 400 new samples. Mean log-likelihood per point: -4.61 (one Gaussian), -3.63 (mixture of three).

The Kullback–Leibler Divergence

Mismatch of a model q with the true p:

$$\mathrm{KL}(p \,\|\, q) = \mathbb{E}_{x\sim p}\Big[\log \frac{p(x)}{q(x)}\Big]$$

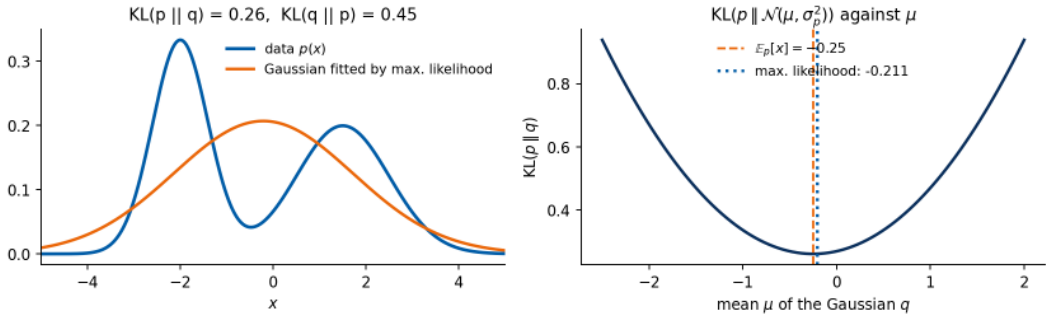
≥ 0, $= 0$ only if $q = p$, not symmetric (so not a distance). With $-\log q(x)$ the surprise of seeing x under q: the average extra surprise when data from p are described by q.

With $q = p_\theta$, split the logarithm:

$$\mathrm{KL}(p \,\|\, p_\theta) = \underbrace{\mathbb{E}_p[\log p]}_{\text{no } \theta} - \underbrace{\mathbb{E}_p[\log p_\theta]}_{\approx \ell(\theta)/n}$$

maximum likelihood = minimum $\mathrm{KL}(p \,\|\, p_\theta)$

Maximum Likelihood and the KL Divergence

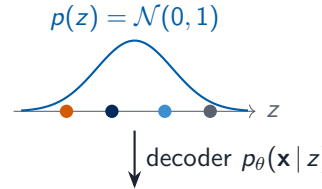


One Gaussian (width fixed at σ_p, the std of p) fitted to the two-peaked p: the KL is smallest at $\mu = \mathbb{E}_p[x] = -0.25$; maximum likelihood on 10^4 samples gives -0.21.

Latent-Variable Models

A simple hidden $\mathbf{z}$ and a map to the data:

$$p_\theta(\mathbf{x}) = \int p_\theta(\mathbf{x} \mid \mathbf{z})\, p(\mathbf{z})\, d\mathbf{z}$$



- **generate**: draw $\mathbf{z} \sim p(\mathbf{z})$, then $\mathbf{x} \sim p_\theta(\mathbf{x} \mid \mathbf{z})$
- Gaussian mixture: $z \in \{1, \ldots, K\}$ with $p(z = k) = \pi_k$, $p(\mathbf{x} \mid z = k) = \mathcal{N}(\boldsymbol{\mu}_k, \Sigma_k)$
- VAE: $\mathbf{z} \in \mathbb{R}^d$, $p(\mathbf{z}) = \mathcal{N}(0, I)$, and a neural network (the decoder) sets $p_\theta(\mathbf{x} \mid \mathbf{z})$

each z gives a small cloud around a point of a curve in data space

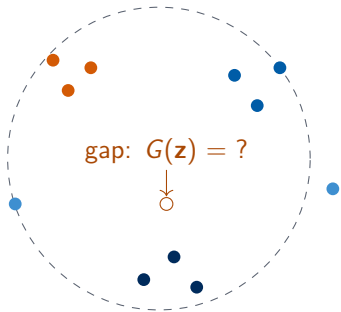
3

Variational Autoencoders

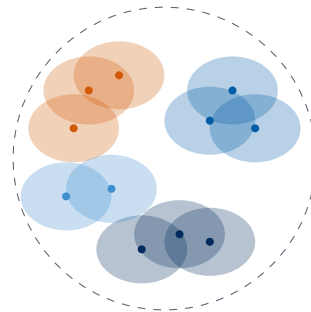
JAGIELLONIAN UNIVERSITY
IN KRAKÓW

Latent Points and Latent Clouds

autoencoder: one point per input



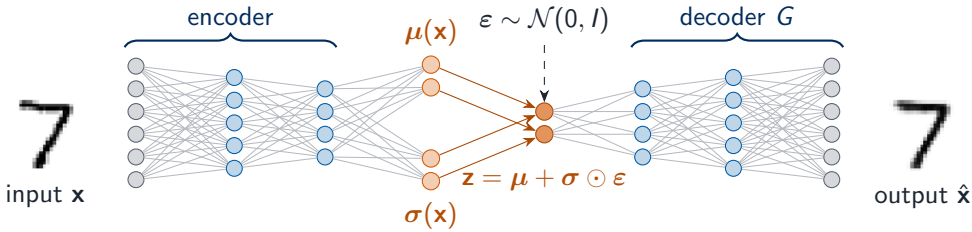
VAE: a Gaussian cloud per input



dashed circle: where $\mathcal{N}(0, I)$ puts most of its probability

Colours: four kinds of input. The VAE encoder gives each input a cloud $\mathcal{N}(\boldsymbol{\mu}(\mathbf{x}), \operatorname{diag} \boldsymbol{\sigma}^2(\mathbf{x}))$; the clouds overlap and fill $\mathcal{N}(0, I)$, so a random $\mathbf{z} \sim \mathcal{N}(0, I)$ lands where the decoder was trained.

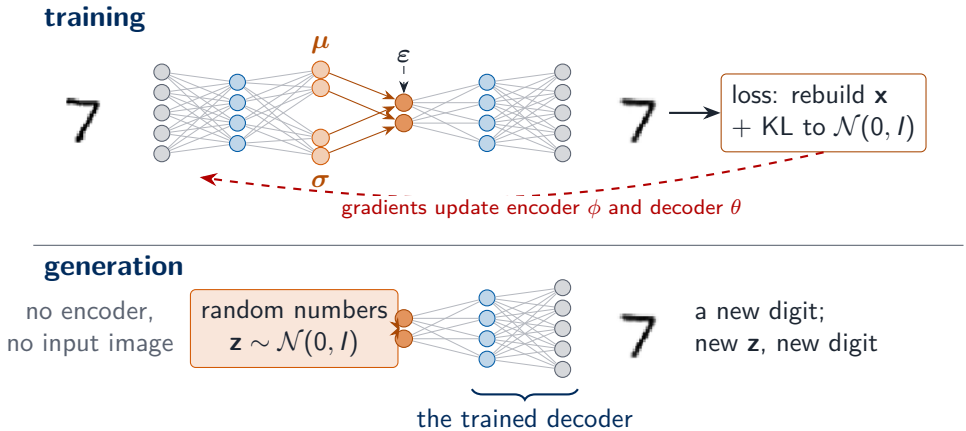
Variational Autoencoder



- an autoencoder maps an input to one *point* $\mathbf{z}$; a VAE maps it to a Gaussian *cloud* centred at $\mu(\mathbf{x})$ with widths $\sigma(\mathbf{x})$ (one μ_k, σ_k for each of the d latent coordinates)
- during training a random point of the cloud is decoded, so every point near $\mu(\mathbf{x})$ must decode to something like $\mathbf{x}$
- to *generate*, skip the encoder: draw $\mathbf{z} \sim \mathcal{N}(0, I)$ and compute $G(\mathbf{z})$

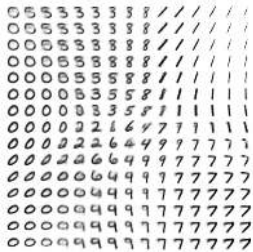
D. P. Kingma, M. Welling, *Auto-Encoding Variational Bayes*, ICLR 2014, arXiv:1312.6114

Training and Sampling a VAE

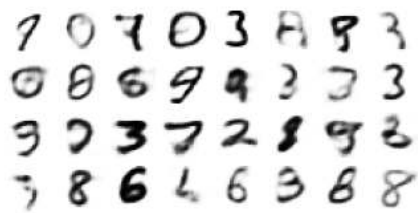


The KL term has placed the training clouds inside $\mathcal{N}(0, I)$, so random numbers drawn from it land where the decoder has learned to produce digits.

New Digits from the VAE



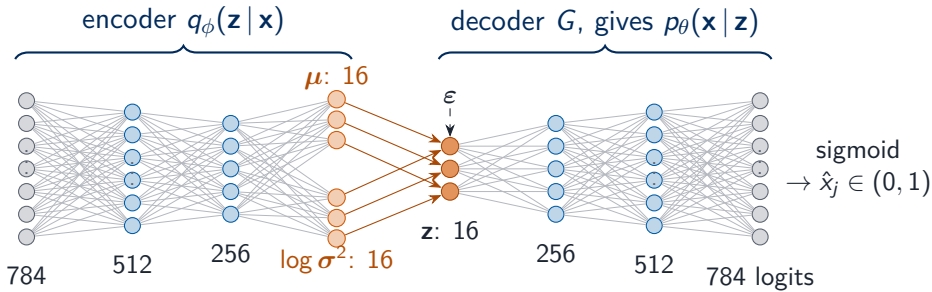
$d = 2$: images decoded from a grid of $\mathbf{z}$ over $[-2.5, 2.5]^2$



$d = 16$: 32 new digits, each from a random $\mathbf{z} \sim \mathcal{N}(0, I)$

- continuous latent space: neighbouring $\mathbf{z}$ give similar digits
- new, not memorised: on average a generated digit is as far from its nearest training image (Euclidean distance in pixel space, 4.73) as a real test digit the network never saw (4.03); somewhat blurry, a known property of VAEs

The VAE on MNIST: Layers



Dense layers with ReLU (a few nodes drawn per layer; the numbers give the true sizes); logits: the outputs before the sigmoid. ϕ: weights of the encoder, θ: of the decoder. The encoder outputs $\log \sigma^2$, which can take any sign; $d = 2$ or 16. Trained 20 epochs with Adam on the 60 000 training digits, without labels.

The Decoder Defines $p_\theta(\mathbf{x} \mid \mathbf{z})$

The decoder ends with a sigmoid, $\hat{\mathbf{x}} = G(\mathbf{z}) \in (0,1)^{784}$ (its last layer outputs *logits*, the numbers before the sigmoid); $\hat{x}_j$ is the probability that pixel j is ink. For black/white pixels $x_j \in \{0,1\}$, drawn independently given $\mathbf{z}$:

$$p_\theta(\mathbf{x} \mid \mathbf{z}) = \prod_{j=1}^{784} \hat{x}_j^{\,x_j} \, (1-\hat{x}_j)^{1-x_j} \quad \Longrightarrow \quad -\log p_\theta(\mathbf{x} \mid \mathbf{z}) = -\sum_{j=1}^{784} \Big[x_j \log \hat{x}_j + (1-x_j) \log(1-\hat{x}_j) \Big]$$

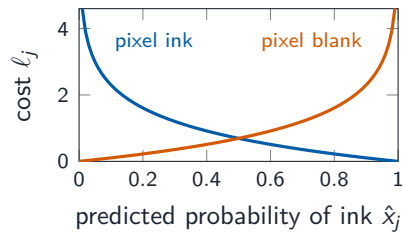
the reconstruction loss (binary cross-entropy) is minus a log-likelihood. MNIST pixels are grey, $x_j \in [0,1]$: the same formula is still used as the loss.

Binary Cross-Entropy

Pixel j is ink ($x_j = 1$) or blank ($x_j = 0$); the decoder gives the probability of ink $\hat{x}_j$. The cost of pixel j is minus the log-probability given to what it actually is; the reconstruction loss is $\sum_{j=1}^{784} \ell_j$:

$$\ell_j = -\big[x_j \log \hat{x}_j + (1 - x_j) \log(1 - \hat{x}_j)\big] = \begin{cases} -\log \hat{x}_j, & x_j = 1 \text{ (ink)} \\ -\log(1 - \hat{x}_j), & x_j = 0 \text{ (blank)} \end{cases}$$

- $\ell_j = 0$ only if the decoder is certain and right; it grows without bound if it is confident and wrong
- *binary*: two outcomes per pixel; *cross-entropy*: the surprise $-\log$ of the true outcome (classification loss, two classes)



Gaussian Pixels and Independent Pixels

Gaussian pixels give the squared error

$p_\theta(\mathbf{x} \mid \mathbf{z}) = \mathcal{N}(\mathbf{x}; G(\mathbf{z}), \sigma_x^2 I)$:

$$-\log p_\theta = \frac{\|\mathbf{x} - G(\mathbf{z})\|^2}{2\sigma_x^2} + \text{const}$$

Independent pixels

given $\mathbf{z}$, the product treats pixels as unrelated: all correlations between pixels must come from $\mathbf{z}$.

The Evidence Lower Bound

Maximum likelihood needs $\log p_\theta(\mathbf{x}) = \log \int p_\theta(\mathbf{x} \mid \mathbf{z})\, p(\mathbf{z})\, d\mathbf{z}$: an integral over all $\mathbf{z}$, out of reach. The encoder $q_\phi(\mathbf{z} \mid \mathbf{x})$ approximates the posterior $p_\theta(\mathbf{z} \mid \mathbf{x})$. Since $p_\theta(\mathbf{x}) = p_\theta(\mathbf{x}, \mathbf{z}) / p_\theta(\mathbf{z} \mid \mathbf{x})$ for every $\mathbf{z}$, average log of it over $\mathbf{z} \sim q_\phi$ and multiply inside by q_ϕ / q_ϕ:

$$\log p_\theta(\mathbf{x}) = \mathbb{E}_{q_\phi}\Big[\log \frac{p_\theta(\mathbf{x}, \mathbf{z})}{q_\phi(\mathbf{z} \mid \mathbf{x})}\Big] + \mathbb{E}_{q_\phi}\Big[\log \frac{q_\phi(\mathbf{z} \mid \mathbf{x})}{p_\theta(\mathbf{z} \mid \mathbf{x})}\Big]$$

With $p_\theta(\mathbf{x}, \mathbf{z}) = p_\theta(\mathbf{x} \mid \mathbf{z})\, p(\mathbf{z})$:

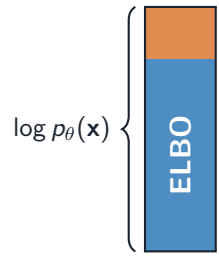
$$\log p_\theta(\mathbf{x}) = \underbrace{\mathbb{E}_{q_\phi}\big[\log p_\theta(\mathbf{x} \mid \mathbf{z})\big] - \mathrm{KL}(q_\phi \,\|\, p(\mathbf{z}))}_{\text{ELBO}} + \underbrace{\mathrm{KL}(q_\phi \,\|\, p_\theta(\mathbf{z} \mid \mathbf{x}))}_{\geq 0}$$

The VAE Loss

Per input $\mathbf{x}$, with one sample $\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\varepsilon}$:

$$\mathcal{L}(\mathbf{x}) = \underbrace{-\log p_\theta(\mathbf{x} \mid \mathbf{z})}_{\text{reconstruction (BCE)}} + \underbrace{\tfrac{1}{2} \sum_{k=1}^{d} \left(\mu_k^2 + \sigma_k^2 - \ln \sigma_k^2 - 1\right)}_{\mathrm{KL}(\mathcal{N}(\boldsymbol{\mu}, \mathrm{diag}\, \boldsymbol{\sigma}^2) \,\|\, \mathcal{N}(0, I))}$$

- $\mathcal{L} = -\text{ELBO}$; since $\mathrm{KL} \geq 0$, $-\mathcal{L} \leq \log p_\theta(\mathbf{x})$: minimising $\mathcal{L}$ raises a lower bound on the log-likelihood
- each KL term is 0 only at $\mu_k = 0$, $\sigma_k = 1$
- the two terms pull apart: reconstruction wants small clouds far from each other, the KL pulls every cloud to $\mathcal{N}(0, I)$; the balance: overlapping clouds that fill $\mathcal{N}(0, I)$ without gaps



$d = 16$ VAE, mean over the training digits: $\mathcal{L} = 99.9 = 80.3$ (reconstruction) $+ 19.6$ (KL)

The Reparametrisation Trick

Training needs $\partial\mathcal{L}/\partial\boldsymbol{\mu}$ and $\partial\mathcal{L}/\partial\boldsymbol{\sigma}$, but $\mathbf{z}$ is *drawn at random*: "draw $\mathbf{z}$ from $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2)$" has no derivative with respect to $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$.

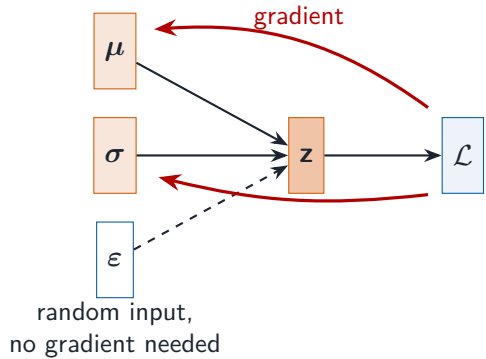
Move the randomness out of the network:

$$\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I)$$

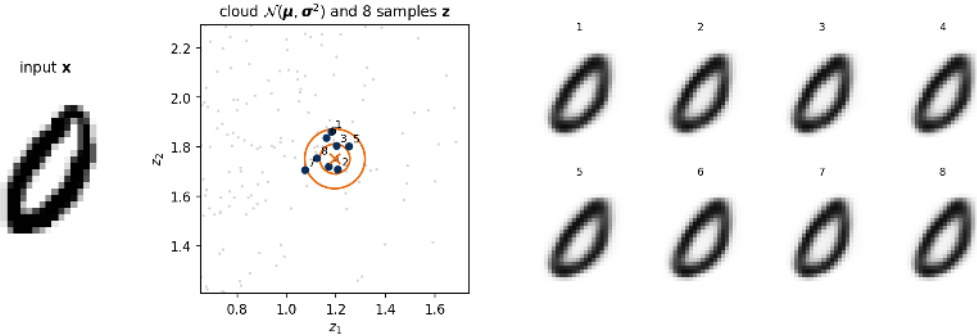
ε is an input that does not depend on the weights; $\mathbf{z}$ is an ordinary function of $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$:

$$\frac{\partial z_k}{\partial \mu_k} = 1, \qquad \frac{\partial z_k}{\partial \sigma_k} = \varepsilon_k,$$

and backpropagation runs from the loss through the decoder, $\mathbf{z}$ and the encoder.



Samples from the Cloud of One Input



One test digit (the 0 that the $d = 2$ VAE rebuilds best; with two latent numbers some unusual digits are rebuilt as another digit). The encoder gives its cloud: centre $\boldsymbol{\mu}$ (cross), widths $\boldsymbol{\sigma}$ (ellipses: 1 and 2σ). Eight noises $\boldsymbol{\varepsilon} \sim \mathcal{N}(0, I)$ give $\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\varepsilon}$ (dots); the decoder turns each into a version of the input. In training, one such $\mathbf{z}$ per input is decoded and compared with $\mathbf{x}$; autograd gives $\partial z_1/\partial \mu_1 = 1$, $\partial z_1/\partial \sigma_1 = \varepsilon_1$.

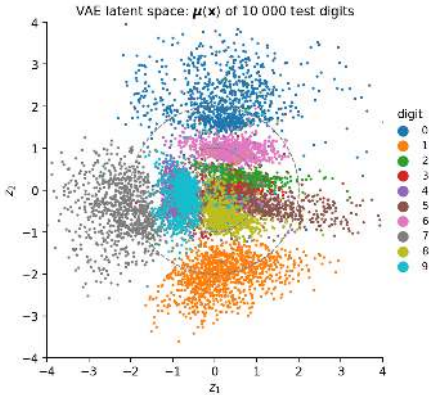
Sampling a VAE

One draw is two random choices, *ancestral sampling* of the model $p_\theta(\mathbf{x}) = \int p_\theta(\mathbf{x} \mid \mathbf{z})\, p(\mathbf{z})\, d\mathbf{z}$:

$$\mathbf{z} \sim \mathcal{N}(0, I), \qquad \mathbf{x} \sim p_\theta(\mathbf{x} \mid \mathbf{z}) \;\; \text{(each pixel ink with probability } \hat{x}_j(\mathbf{z})\text{)}$$

- in practice we show the *mean image* $\hat{\mathbf{x}}(\mathbf{z}) = G(\mathbf{z})$, the pixel probabilities, rather than one random draw of the pixels: the grey levels are probabilities of ink
- $\mathbf{z}$ carries the content (which digit, its slant and thickness); the pixel draw only adds independent noise
- **reconstruction** (encode $\mathbf{x}$, decode) reproduces a given image; **generation** (draw $\mathbf{z}$, decode) makes a new one
- **interpolation**: encode two digits to $\mathbf{z}_A$, $\mathbf{z}_B$ and decode $(1-t)\,\mathbf{z}_A + t\,\mathbf{z}_B$: one digit turns smoothly into the other, because the latent space is filled without gaps

The Latent Space of a VAE



Latent representation: the encoder compresses $\mathbf{x} \in \mathbb{R}^{784}$ into $\mathbf{z} \in \mathbb{R}^d$, coordinates in which similar images are close.

VAE with $d = 2$, trained without labels; each point: the centre $\boldsymbol{\mu}(\mathbf{x})$ of one test digit's cloud, coloured by its label.

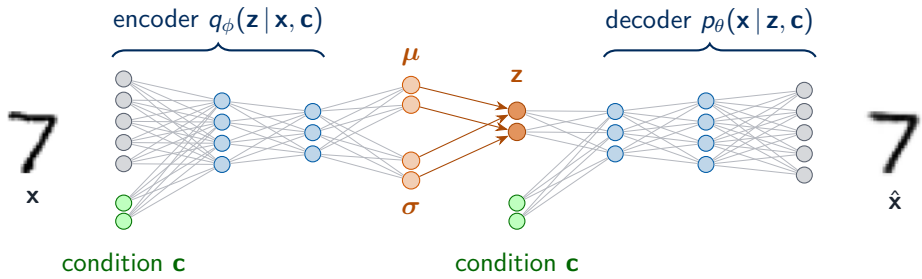
- digits of one class gather: the latent space organises the data
- 80% of the centres lie within radius 2 (dashed: radius 1, 2), against 86% for $\mathcal{N}(0, I)$; classes 0, 1, 7 sit at the edge

4

Conditional Generation

JAGIELLONIAN UNIVERSITY
IN KRAKOW

Conditional VAE



$\mathbf{c}$ is an extra input of *both* networks, $q_\phi(\mathbf{z} \mid \mathbf{x}, \mathbf{c})$ and $p_\theta(\mathbf{x} \mid \mathbf{z}, \mathbf{c})$; the model is $p_\theta(\mathbf{x} \mid \mathbf{c}) = \int p_\theta(\mathbf{x} \mid \mathbf{z}, \mathbf{c})\, p(\mathbf{z})\, d\mathbf{z}$, with the same loss as the VAE. To generate: fix $\mathbf{c}$, draw $\mathbf{z} \sim \mathcal{N}(0, I)$, compute $G(\mathbf{z}, \mathbf{c})$.

K. Sohn, H. Lee, X. Yan, *Learning Structured Output Representation using Deep Conditional Generative Models*, NeurIPS 2015, NeurIPS proceedings

Condition and Latent Vector

- $\mathbf{c}$ tells the decoder *what* to draw. Every bit stored in $\mathbf{z}$ costs KL, while what $\mathbf{c}$ supplies is free, so $\mathbf{z}$ keeps only what $\mathbf{c}$ does not say: the *style* (slant, thickness, size)

- a class k enters *one-hot*, $\mathbf{e}_k = (0, \ldots, 0, 1, 0, \ldots, 0)$ with the 1 at position k, not as the number k, which would impose an order on the classes; physical parameters such as (T, h) enter as numbers

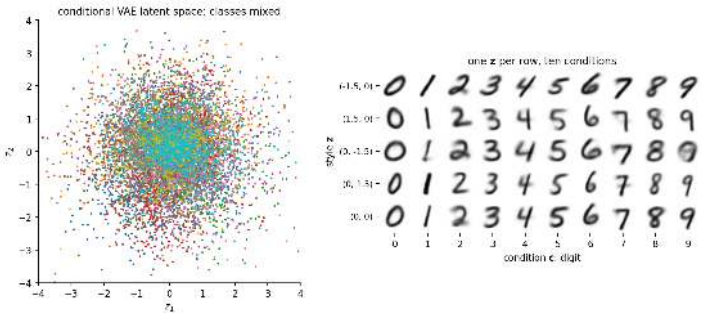
Sampling a Conditional VAE

The model is $p_\theta(\mathbf{x} \mid \mathbf{c}) = \int p_\theta(\mathbf{x} \mid \mathbf{z}, \mathbf{c})\, p(\mathbf{z})\, d\mathbf{z}$. Sampling: we *choose* $\mathbf{c}$ and *draw* $\mathbf{z}$:

$$\mathbf{c} \text{ given (what)}, \qquad \mathbf{z} \sim \mathcal{N}(0, I) \text{ (how)}, \qquad \hat{\mathbf{x}} = G(\mathbf{z}, \mathbf{c})$$

- one $\mathbf{c}$, many $\mathbf{z}$: many instances of the same class, different in style (digits 7 in many handwritings; cells of one type)
- one $\mathbf{z}$, many $\mathbf{c}$: the same style for every class (the rows of the digit grid on the next slides keep slant and thickness)
- continuous conditions work the same way: fix the temperature T and draw $\mathbf{z}$ to get spin configurations at that T
- the condition must be known for every training image: a conditional VAE needs labels; what the labels do not describe is left to $\mathbf{z}$

Latent Space of the Conditional VAE



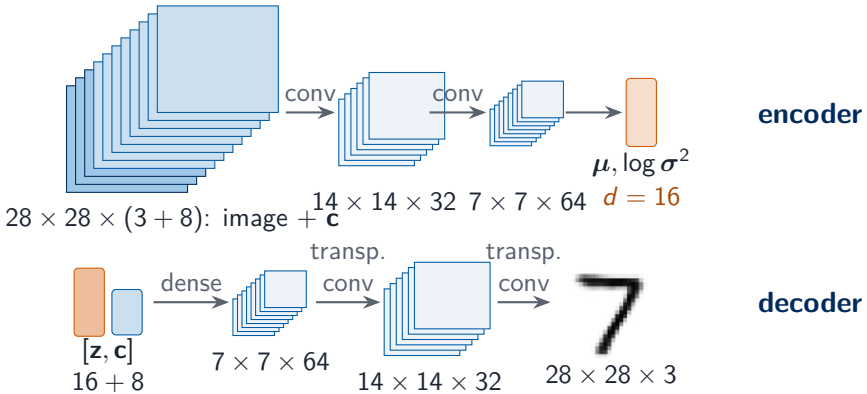
Conditional VAE with $d = 2$, label given as $\mathbf{c}$. Left: the centres $\boldsymbol{\mu}(\mathbf{x}, \mathbf{c})$ of the test digits, coloured by digit, are mixed: the position no longer tells the class. Right: one fixed $\mathbf{z}$ per row, the ten conditions along the row: the digit changes, slant and thickness stay. **c chooses what, z chooses how.**

Conditional VAE on MNIST



- each row: $\mathbf{c}$ = one digit, 0 to 9; each column: one random $\mathbf{z} \sim \mathcal{N}(0, I)$, the *same* in every row
- along a column the style stays and the digit changes: $\mathbf{z}$ carries the style, $\mathbf{c}$ the identity
- the generated digits are checked with a classifier of real digits

The Condition in a Convolutional Network



BloodMNIST: the one-hot cell type enters as 8 extra constant channels (1 for the requested type, 0 otherwise); stride-2 convolutions halve the size, transposed convolutions double it.

Generating a Chosen Digit

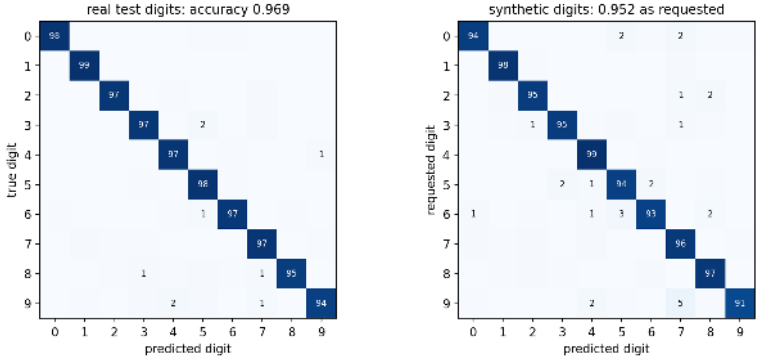
Only the decoder: condition $\mathbf{c} = \mathbf{e}_k$ (one-hot digit k), random style $\mathbf{z} \sim \mathcal{N}(0, I)$, image sigmoid $\big(G(\mathbf{z}, \mathbf{c})\big)$.

```
def generate_digit(k, n=10):
    c = F.one_hot(torch.full((n,), k),
                  10).float()
    z = torch.randn(n, cvae.d)
    with torch.no_grad():
        return torch.sigmoid(
            cvae.decode(z, c))
```

Row k: `generate_digit(k, 12)`; each column a new $\mathbf{z}$.

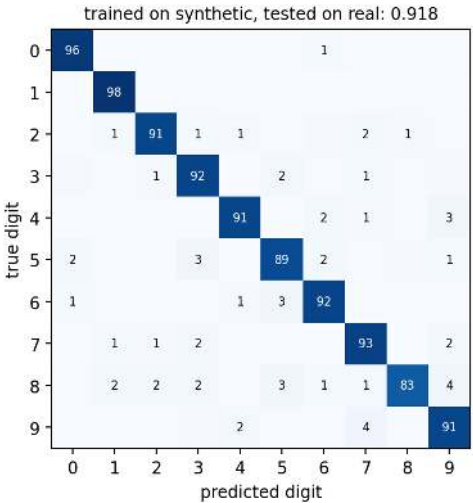


Classifier Check of Synthetic Digits



Classifier $784 \to 256 \to 10$, trained on real digits only. Left: on the real test digits (row: true digit). Right: on 1000 generated digits per class (row: requested digit); entries in % of the row. Diagonal mean: 0.95; the confusions resemble those of real digits.

Training on Synthetic Digits Only



The same classifier, trained on 60 000 *generated* digits (6000 per class, labels = requested digits), tested on the real test digits:

- trained on real digits: accuracy 0.969
- trained on synthetic digits: accuracy 0.918

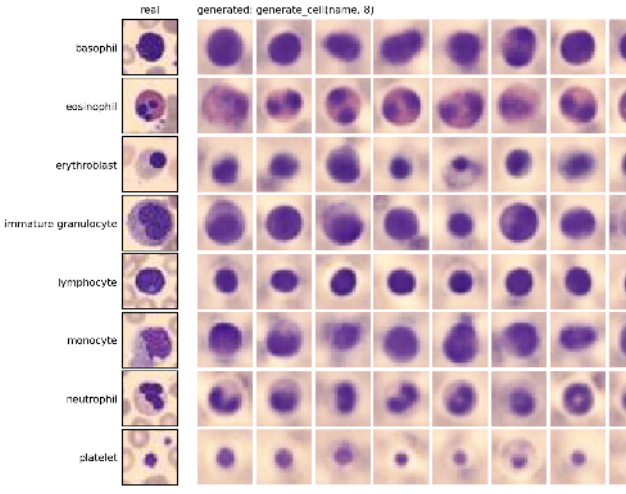
The gap measures what the generator misses of the real distribution: styles it never produces, and blurred strokes.

Generating a Cell of a Chosen Type

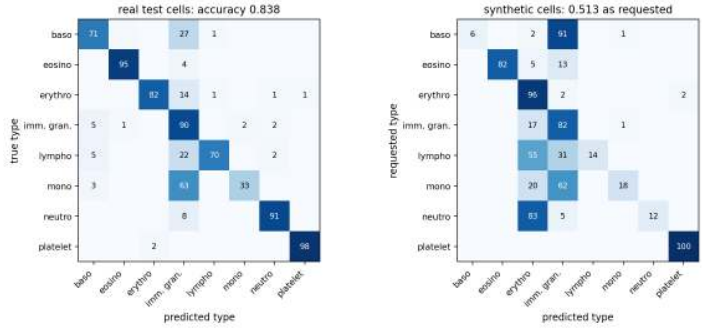
The same recipe, with the cell type as condition and an RGB output:

```
def generate_cell(name, n=8):
    k = names.index(name)
    c = F.one_hot(torch.full((n,),
                  8).float()
    z = torch.randn(n, bcvae.d)
    with torch.no_grad():
        return torch.sigmoid(
            bcvae.decode(z, c))
```

First column: one real cell of the type; then 8 generated.

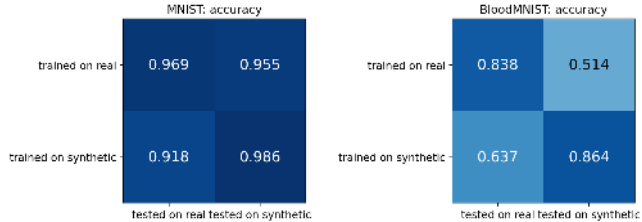


Classifier Check of Synthetic Cells



CNN trained on real cells only. Left: real test cells (accuracy 0.84). Right: 500 generated cells per requested type (diagonal mean 0.51). Platelets (100%) and erythroblasts (96%), told apart by size, come out right; generated basophils are taken for immature granulocytes (91%), neutrophils for erythroblasts (83%): the VAE blurs the shape of the nucleus.

Training and Testing on Real and Synthetic Data



- **trained on real, tested on synthetic:** does the generator draw the requested class?
- **trained on synthetic, tested on real:** do the synthetic data carry what is needed to recognise real instances?
- MNIST: synthetic digits almost replace real ones (0.92 against 0.97); blood cells: trained on synthetic, 0.64 on real cells (0.84 with real training) but 0.86 on synthetic: types easy to tell apart, but not the real ones

5

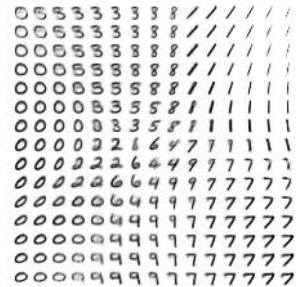
Disentangled Latent Features

JAGIELLONIAN UNIVERSITY
IN KRAKOW

Latent Coordinates and Features

Each latent coordinate z_k is a direction; moving along it, the others fixed, changes the decoded image (a traversal).

- in the $d = 2$ VAE (right) moving z_1 or z_2 changes several properties at once: the digit identity, its slant, its width
- **disentangled**: each coordinate responds to one independent property of the data and ignores the others
- the goal: one latent node $\leftrightarrow$ one distinct feature, found without labels (unlike the condition $\mathbf{c}$ of a cVAE)



$d = 2$ VAE: decoded images on a grid of (z_1, z_2)

The β-VAE Loss

One weight more in the VAE loss:

$$\mathcal{L}_\beta(\mathbf{x}) = \underbrace{\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}\big[-\log p_\theta(\mathbf{x}\,|\,\mathbf{z})\big]}_{\text{reconstruction}} + \beta\,\underbrace{\mathrm{KL}\big(q_\phi(\mathbf{z}\,|\,\mathbf{x})\,\|\,\mathcal{N}(0,I)\big)}_{\sum_k \frac{1}{2}(\mu_k^2+\sigma_k^2-\ln\sigma_k^2-1)}, \qquad \beta \geq 1$$

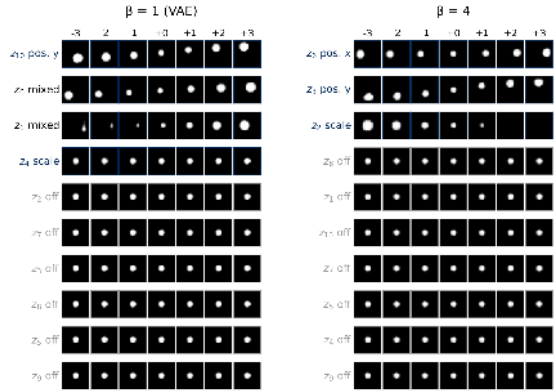
- $\beta = 1$: the VAE, minus the ELBO; $\beta > 1$: information stored in $\mathbf{z}$ costs more
- the prior $\mathcal{N}(0, I)$ has *independent* coordinates; a stronger pull towards it makes each kept coordinate carry one independent factor of variation
- a coordinate with $\mu_k \approx 0$, $\sigma_k \approx 1$ for every input (KL ≈ 0) carries nothing: the KL per coordinate shows which are used
- the price: blurrier reconstructions and samples; β trades sharpness for interpretable coordinates

I. Higgins et al., β-VAE, ICLR 2017, openreview.net/forum?id=Sy2fzU9gl

Sampling a β-VAE

Generation as for any VAE: $\mathbf{z} \sim \mathcal{N}(0, I)$, decode. What changes is the *control*:

- fix $\mathbf{z}$ and move one coordinate z_k: only one property changes (position, size, rotation; pose, smile)
- no labels needed: the properties are discovered, and named afterwards by looking at the traversals
- a cVAE controls only labelled properties; a β-VAE finds unlabelled ones, without saying which coordinate is which



VAE, Conditional VAE and β-VAE

	VAE	conditional VAE	β-VAE
trained on	images $\mathbf{x}$	pairs $(\mathbf{x}, \mathbf{c})$	images $\mathbf{x}$
loss	reconstruction + KL	reconstruction + KL	reconstruction + β KL
to sample	$\mathbf{z} \sim \mathcal{N}(0, I)$	choose $\mathbf{c}$, $\mathbf{z} \sim \mathcal{N}(0, I)$	$\mathbf{z} \sim \mathcal{N}(0, I)$, or move one z_k
$\mathbf{z}$ controls	everything, mixed	what $\mathbf{c}$ does not say (style)	separate properties, one per coordinate
labels	not needed	needed	not needed
use	new samples	samples of a chosen class or T	finding and controlling factors

Summary

- a **generative model** learns the distribution $p(\mathbf{x})$ of the data from samples, and draws new samples from it
- **maximum likelihood**: choose the parameters that make the observed data most probable; it minimises the KL divergence to the data
- **VAE**: a latent space with a known distribution, $\mathcal{N}(0, I)$; trained on reconstruction + KL; decode random $\mathbf{z}$ to generate
- **conditional VAE**: generates what we ask for, a digit or a cell type
- β-**VAE**: one latent coordinate per feature, found without labels: a tool for interpretation

Tutorial: Hands-on

Hands-on

- train the VAE without the KL term and generate from $\mathcal{N}(0, I)$: what goes wrong?
- generate digits with a conditional VAE and walk along one latent coordinate: which property of the digit changes?
- train a β-VAE with $\beta = 1$ and $\beta = 4$ and compare the latent traversals

Next Lecture: Adversarial and Diffusion Models

- generative adversarial networks: a generator trained against a discriminator, without a likelihood
- diffusion models: learning to remove noise
- spin configurations of the Ising, XY and spin-glass models generated at a chosen temperature and checked against Monte Carlo

Thank You!

Questions, project ideas, feedback: marcin.plodzien@uj.edu.pl

Website · GitHub · Google Scholar



